

Loom Walkthrough Guide

Presentation-ready recording guide for explaining RoleRAG Interviewer step by step. Use this as your speaking script, screen flow, and topic checklist while recording.

1. Recommended Video Goal

Record a clear 8 to 12 minute walkthrough that explains what the project does, why it exists, how the architecture works, how the user flow works, and what technical decisions make it interesting.

Best approach: keep the video practical. Show the UI first, then reveal the code and architecture behind it.

2. Recording Structure

0:00 - 0:45 Introduction

Introduce the project, problem statement, and the main idea.

0:45 - 2:00 What the Application Does

Show the UI and explain the interview lifecycle at a high level.

2:00 - 4:00 System Architecture

Explain frontend, backend, services, database, and Chroma-based retrieval.

4:00 - 6:30 Backend Deep Dive

Walk through endpoints, resume parsing, RAG pipeline, question generation, and evaluation logic.

6:30 - 8:00 Frontend Deep Dive

Show the components and explain how state flows during the interview.

8:00 - 9:30 Strengths, Trade-offs, and Limitations

Talk honestly about what is good now and what you would improve next.

9:30 - 10:30 Closing

Summarize the value of the system and what makes it different from a basic prompt app.

3. Opening Script

What to say

"This project is called RoleRAG Interviewer. It is a full-stack interview system that takes a candidate resume, retrieves role-specific knowledge-base content, generates grounded technical questions, evaluates answers, and produces a final summary. The main goal is to make technical interviews more relevant than generic question generation."

What to show

Open the app home screen first, then briefly show the repository structure in the IDE.

4. Problem Statement Talking Points

- Generic interview bots ask shallow or repeated questions.
- They usually ignore the candidate's real resume context.
- They often have no grounding in actual role knowledge.
- This project tries to fix that by combining resume parsing with role-specific RAG retrieval.

What to say

"Instead of asking generic ML or backend questions, this system first understands the candidate profile and then retrieves role-specific context from a knowledge base. That makes the generated questions more relevant, more grounded, and easier to justify."

5. Demo Flow Script

What to show

1. Show the role dropdown.
2. Show resume upload input.
3. Explain that the backend extracts text and creates a profile.
4. Show the first interview question and source grounding area.
5. Show how answers are evaluated with score, strengths, gaps, and follow-up.
6. Show the final summary view after interview completion.

What to say

"From the UI, the user selects a role and uploads a resume. Once the interview starts, the system builds a structured profile, retrieves relevant role knowledge, asks a grounded question, and continues adaptively based on the candidate's answers. Each turn is stored along with retrieved context and evaluation output."

6. Architecture Explanation Script

What to show

Open docs/ARCHITECTURE.md and the main folder structure.

What to say

"The architecture is split into clear layers. The React frontend handles user interaction and renders the interview lifecycle. The FastAPI backend exposes endpoints for starting interviews, submitting answers, resetting sessions, and fetching summaries. The service layer handles resume parsing, RAG retrieval, and LLM-based reasoning. SQLAlchemy stores sessions and turns, while ChromaDB stores the vectorized knowledge base."

7. Backend Walkthrough Script

Main Entry

Show

```
backend/app/main.py
```

Say

"This file initializes FastAPI, enables CORS for the frontend, includes the interview router, and creates the database tables on startup."

API Routes

Show

```
backend/app/api/interview.py
```

Say

"This is the main orchestration layer. It handles KB ingestion, interview start, answer submission, summary fetch, and reset. The start endpoint creates the first question, while the answer endpoint stores the response, evaluates it, and either generates the next question or finalizes the session summary."

Database Model

Show

```
backend/app/models/db.py
```

Say

"The data model is intentionally simple: one table for interview sessions and one table for interview turns. This preserves the trace of question, answer, retrieved context, and evaluation for each step."

8. Resume Parsing Script

What to show

```
backend/app/services/resume.py
```

What to say

"The resume service first extracts raw text from PDF or TXT. Then it tries to build a structured profile. If an OpenAI API key is available, it uses the model to extract clean JSON. If not, it falls back to deterministic parsing using section detection, regex patterns, skill hints, and domain inference. That makes the system more resilient during demos."

9. RAG Pipeline Script

What to show

```
backend/app/services/rag.py and backend/data/kb_docs/README.md
```

What to say

"The RAG service handles knowledge-base ingestion and retrieval. It reads role-specific documents, chunks them into roughly 950-word windows with overlap, embeds them using all-MiniLM-L6-v2, and stores them in Chroma. During the interview, it builds queries from the role, skills, projects, domains, and even the latest answer, then retrieves the most relevant chunks."

Mention why separate collections are used: it keeps retrieval focused and avoids contamination across roles.

10. LLM Logic Script

What to show

`backend/app/services/llm.py`

What to say

"This file handles three important tasks: question generation, answer evaluation, and final summary creation. The prompt for question generation includes the role, extracted resume profile, retrieved knowledge-base context, previous questions, and the latest answer. That helps the model produce grounded and non-repetitive questions. There is also fallback logic so the system can still operate without OpenAI."

11. Frontend Walkthrough Script

What to show

Open `frontend/src/App.jsx` first, then show the components folder.

What to say

"The frontend is centered around one main stateful component. It tracks the session id, extracted profile, current question, current answer, historical turns, summary, loading state, and error state. The app sends form data to start the interview and JSON for each answer submission. The UI is broken into smaller components like candidate entry, question card, turn log, evaluation card, stepper, and summary view."

12. Important Technical Decisions to Explain

- Why the app uses role-specific Chroma collections.
- Why it keeps turn-level context and evaluation in the database.
- Why it supports fallback logic without an OpenAI key.
- Why chunk size is relatively large for textbook-style sources.
- Why FastAPI + React is a good choice for quick, well-structured prototyping.

13. Strengths and Limitations Script

What to say

"One strength of this project is that it is not just a single prompt chain. It has a complete lifecycle with storage, adaptive questioning, and retrieval grounding. Another strength is the offline fallback behavior, which makes the demo more dependable. On the limitation side, the current database is SQLite, ingestion is synchronous, and the evaluation heuristics are simpler when OpenAI is unavailable."

14. Closing Script

What to say

"Overall, RoleRAG Interviewer shows how retrieval, personalization, adaptive logic, and structured storage can be combined into a practical interview platform. The main value is that the questions are both candidate-aware and knowledge-grounded, which makes the interview experience more relevant and more explainable."

What to show

End on the summary screen or the repository README, depending on whether you want to finish with product value or code structure.

15. Recording Checklist

- Open app UI before starting the narration.
- Keep the IDE ready with backend and frontend files.
- Have one sample resume ready.
- Mention supported roles clearly.
- Explain where RAG is used and why.
- Show at least one endpoint file.
- Show at least one service file.
- Show the turn log and summary UI.
- Call out fallback behavior without API key.
- End with strengths and future improvements.